

PyTest: структура, assert и fixture

Цели видео

- ✓ Научиться писать и запускать простые тесты
- ✓ Познакомиться с разными способами проверки в тестах
- ✓ Научиться создавать fixtures

Первая задача на Big Data-проекте

Доработать PySpark-код, который рассчитывает распределение дистанций поездок по районам:

- количество заказов
- минимальная дистанция
- максимальная дистанция
- средняя дистанция

Первая задача на Big Data-проекте

```
taxiDF = spark.read.load("yellow_taxi_jan_25_2018")
taxiZonesDF = spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)

taxiDF.join(taxiZonesDF, col("LocID") == col("LocID")) \
    .groupBy("Borough") \
    .agg(
        count("*").alias("total trips"),
        min("trip_distance").alias("min distance"),
        round(mean("trip_distance"), 2).alias("mean distance"),
        max("trip_distance").alias("max distance")
    ) \
    .write.format("csv") \
    .option("header", "true") \
    .option("sep", "\t") \
    .save("popular_borough.csv")
```

С чего начать задачу по доработке витрины?

Код не покрыт тестами

```
taxiDF = spark.read.load("yellow_taxi_jan_25_2018")
taxiZonesDF = spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)

taxiDF.join(taxiZonesDF, col("LocID") == col("LocID")) \
    .groupBy("Borough") \
    .agg(
        count("*").alias("total trips"),
        min("trip_distance").alias("min distance"),
        round(mean("trip_distance"), 2).alias("mean distance"),
        max("trip_distance").alias("max distance")
    ) \
    .write.format("csv") \
    .option("header", "true") \
    .option("sep", "\t") \
    .save("popular_borough.csv")
```

PyTest: структура, assert и fixture

Какие могут быть проблемы?

Ошибки в логике и в названиях колонок

```
taxiDF = spark.read.load("yellow_taxi_jan_25_2018")
taxiZonesDF = spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)

taxiDF.join(taxiZonesDF, col("LocID") == col("LocID")) \
    .groupBy("Borough") \
    .agg(
        count("*").alias("total trips"),
        min("trip_distance").alias("max distance"),
        round(mean("trip_distance"), 2).alias("mean distance"),
        max("trip_distance").alias("min distance")
    ) \
    .write.format("csv") \
    .option("header", "true") \
    .option("sep", "\t") \
    .save("popular_borough.csv")
```

Ошибки в логике и в названиях колонок

```
taxiDF = spark.read.load("yellow_taxi_jan_25_2018")
taxiZonesDF = spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)

taxiDF.join(taxiZonesDF, col("LocID") == col("LocID")) \
    .groupBy("Borough") \
    .agg(
        count("*").alias("total trips"),
        min("trip_distance").alias("max distance"),
        round(mean("trip_distance"), 2).alias("mean distance"),
        max("trip_distance").alias("min distance")
    ) \
    .write.format("csv") \
    .option("header", "true") \
    .option("sep", "\t") \
    .save("popular_borough.csv")
```

Ошибки в логике и в названиях колонок

```
taxiDF = spark.read.load("yellow_taxi_jan_25_2018")
taxiZonesDF = spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)

taxiDF.join(taxiZonesDF, col("LouuuuucID") == col("LocID")) \
    .groupBy("Barrrrrough") \
    .agg(
        count("*").alias("total trips"),
        min("trip_distance").alias("max distance"),
        round(mean("trip_distance"), 2).alias("mean distance"),
        max("trip_distance").alias("min distance")
    ) \
    .write.format("csv") \
    .option("header", "true") \
    .option("sep", "\t") \
    .save("popular_borough.csv")
```

Ошибки в логике и в названиях колонок

```
taxiDF = spark.read.load("yellow_taxi_jan_25_2018")
taxiZonesDF = spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)

taxiDF.join(taxiZonesDF, col("LouuuuucID") == col("LocID")) \
    .groupBy("Barrrrrough") \
    .agg(
        count("*").alias("total trips"),
        min("trip_distance").alias("max distance"),
        round(mean("trip_distance"), 2).alias("mean distance"),
        max("trip_distance").alias("min distance")
    ) \
    .write.format("csv") \
    .option("header", "true") \
    .option("sep", "\t") \
    .save("popular_borough.csv")
```

Ошибки в параметрах загрузки и выгрузки

```
taxiDF = spark.read.load("yellow_taxi_jan_25_2018")
taxiZonesDF = spark.read.csv("taxi_zones.tsv", header=True, inferSchema=True)

taxiDF.join(taxiZonesDF, col("LocID") == col("LocID")) \
    .groupBy("Borough") \
    .agg(
        count("*").alias("total trips"),
        min("trip_distance").alias("min distance"),
        round(mean("trip_distance"), 2).alias("mean distance"),
        max("trip_distance").alias("max distance")
    ) \
    .write.format("csv") \
    .option("header", "true") \
    .option("sep", "\t") \
    .save("popular_borough.csv")
```

Ошибки в параметрах загрузки и выгрузки

```
taxiDF = spark.read.load("yellow_taxi_jan_25_2018")
taxiZonesDF = spark.read.csv("taxi_zones.tsv", header=True, inferSchema=True)

taxiDF.join(taxiZonesDF, col("LocID") == col("LocID")) \
    .groupBy("Borough") \
    .agg(
        count("*").alias("total trips"),
        min("trip_distance").alias("min distance"),
        round(mean("trip_distance"), 2).alias("mean distance"),
        max("trip_distance").alias("max distance")
    ) \
    .write.format("csv") \
    .option("header", "true") \
    .option("sep", "\t") \
    .save("popular_borough.csv")
```

Код не разбит на блоки

```
taxiDF = spark.read.load("yellow_taxi_jan_25_2018")
taxiZonesDF = spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)

taxiDF.join(taxiZonesDF, col("LocID") == col("LocID")) \
    .groupBy("Borough") \
    .agg(
        count("*").alias("total trips"),
        min("trip_distance").alias("min distance"),
        round(mean("trip_distance"), 2).alias("mean distance"),
        max("trip_distance").alias("max distance")
    ) \
    .write.format("csv") \
    .option("header", "true") \
    .option("sep", "\t") \
    .save("popular_borough.csv")
```

Код не разбит на блоки

```
taxiDF = spark.read.load("yellow_taxi_jan_25_2018")  
taxiZonesDF = spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)
```

```
taxiDF.join(taxiZonesDF, col("LocID") == col("LocID")) \  
    .groupBy("Borough") \  
    .agg(  
        count("*").alias("total trips"),  
        min("trip_distance").alias("min distance"),  
        round(mean("trip_distance"), 2).alias("mean distance"),  
        max("trip_distance").alias("max distance")  
    ) \  
    .write.format("csv") \  
    .option("header", "true") \  
    .option("sep", "\t") \  
    .save("popular_borough.csv")
```

PyTest: структура, assert и fixture

Разбивка кода на блоки

Разбивка кода на блоки

```
taxiDF = spark.read.load("yellow_taxi_jan_25_2018")  
taxiZonesDF = spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)
```

```
taxiDF.join(taxiZonesDF, col("LocID") == col("LocID")) \  
    .groupBy("Borough") \  
    .agg(  
        count("*").alias("total trips"),  
        min("trip_distance").alias("min distance"),  
        round(mean("trip_distance"), 2).alias("mean distance"),  
        max("trip_distance").alias("max distance")  
    ) \
```

```
    .write.format("csv") \  
    .option("header", "true") \  
    .option("sep", "\t") \  
    .save("popular_borough.csv")
```

Разбивка кода на блоки

1. Загрузка данных из источника

```
taxiDF.join(taxiZonesDF, col("LocID") == col("LocID")) \
    .groupBy("Borough") \
    .agg(
        count("*").alias("total trips"),
        min("trip_distance").alias("min distance"),
        round(mean("trip_distance"), 2).alias("mean distance"),
        max("trip_distance").alias("max distance")
    ) \
    .write.format("csv") \
    .option("header", "true") \
    .option("sep", "\t") \
    .save("popular_borough.csv")
```

Разбивка кода на блоки

1. Загрузка данных из источника

2. Обработка данных

```
.write.format("csv") \
.option("header", "true") \
.option("sep", "\t") \
.save("popular_borough.csv")
```

Разбивка кода на блоки

1. Загрузка данных из источника

2. Обработка данных

3. Запись результата в получатель

Разбивка кода на блоки

```
taxiDF = spark.read.load("yellow_taxi_jan_25_2018")  
taxiZonesDF = spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)
```



```
def load_taxi_data(spark):  
    return spark.read.load("yellow_taxi_jan_25_2018")  
  
def load_taxi_zones(spark):  
    return spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)
```

Разбивка кода на блоки

```
taxiDF.join(taxiZonesDF, col("LocID") == col("LocID")) \
    .groupBy("Borough") \
    .agg(
        count("*").alias("total trips"),
        min("trip_distance").alias("min distance"),
        round(mean("trip_distance"), 2).alias("mean distance"),
        max("trip_distance").alias("max distance")
    )
```

Разбивка кода на блоки

```
taxiDF.join(taxiZonesDF, col("LocID") == col("LocID")) \
    .groupBy("Borough") \
    .agg(
        count("*").alias("total trips"),
        min("trip_distance").alias("min distance"),
        round(mean("trip_distance"), 2).alias("mean distance"),
        max("trip_distance").alias("max distance")
    )
```



```
def process_taxi_data(taxiDF, taxiZonesDF):
    return taxiDF.join(taxiZonesDF, col("LocID") == col("LocID")) \
        .groupBy("Borough") \
        .agg(
            count("*").alias("total trips"),
            min("trip_distance").alias("min distance"),
            round(mean("trip_distance"), 2).alias("mean distance"),
            max("trip_distance").alias("max distance")
        )
```

PyTest: структура, assert и fixture

Из чего состоит unit-тест?

Структура unit-теста

1. Название: вход, действие, ожидаемый результат

2. Arrange — подготовка исходных данных

3. Act — запуск тестируемого метода

4. Assert — сопоставление полученного результата с ожидаемым

Структура unit-теста

```
def test_calculate_distribution_by_valid_taxi_data(self):
```

2. Arrange — подготовка исходных данных

3. Act — запуск тестируемого метода

4. Assert — сопоставление полученного результата с ожидаемым

Структура unit-теста

```
def test_calculate_distribution_by_valid_taxi_data(self):  
  
    taxiZonesDF = self.spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)  
    taxiDF = self.spark.read.parquet("yellow_taxi_jan_25_2018")
```

3. Act — запуск тестируемого метода

4. Assert — сопоставление полученного результата с ожидаемым

Структура unit-теста

```
def test_calculate_distribution_by_valid_taxi_data(self):  
  
    taxiZonesDF = self.spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)  
    taxiDF = self.spark.read.parquet("yellow_taxi_jan_25_2018")  
  
    resultDF = process_taxi_data(taxiDF, taxiZonesDF)  
    actualDistribution = resultDF.collect()[0]
```

4. Assert — сопоставление полученного результата с ожидаемым

Структура unit-теста

```
def test_calculate_distribution_by_valid_taxi_data(self):  
  
    taxiZonesDF = self.spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)  
    taxiDF = self.spark.read.parquet("yellow_taxi_jan_25_2018")  
  
    resultDF = process_taxi_data(taxiDF, taxiZonesDF)  
    actualDistribution = resultDF.collect()[0]  
  
    self.assertEqual(actualDistribution["Borough"], "Manhattan")  
    self.assertEqual(actualDistribution["total trips"], 304266)  
    self.assertEqual(actualDistribution["min distance"], 0.0)  
    self.assertEqual(actualDistribution["mean distance"], 2.23)  
    self.assertEqual(actualDistribution["max distance"], 66.0)
```

Как проверить результат?

Assert

```
self.assertEqual(actualDistribution["Borough"], "Manhattan")  
self.assertNotEqual(actualDistribution["total trips"], 0)
```

```
self.assertTrue(actualDistribution["total trips"] > 0)  
self.assertFalse(actualDistribution["mean distance"] < 0)
```

```
self.assertIsNotNone(actualDistribution)  
self.assertIsNone(actualDistribution)
```

```
self.assertIn("Borough", actualDistribution)  
self.assertNotIn("invalid_field", actualDistribution)
```

Assert

```
self.assertIsInstance(actualDistribution["total trips"], int)
self.assertNotIsInstance(actualDistribution["mean distance"], str)
```

```
self.assertGreater(actualDistribution["total trips"], 0)
self.assertLessEqual(actualDistribution["mean distance"], 10)
```

```
self.assertGreaterEqual(actualDistribution["total trips"], 0)
self.assertLess(actualDistribution["mean distance"], 10)
```

```
self.assertRegex(actualDistribution["Borough"], r"^[M]")
self.assertNotRegex(actualDistribution["Borough"], r"^[0-9]")
```

Структура unit-теста

```
def test_calculate_distribution_by_valid_taxi_data(self):  
  
    taxiZonesDF = self.spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)  
    taxiDF = self.spark.read.parquet("yellow_taxi_jan_25_2018")  
  
    resultDF = process_taxi_data(taxiDF, taxiZonesDF)  
    actualDistribution = resultDF.collect()[0]  
  
    self.assertEqual(actualDistribution["Borough"], "Manhattan")  
    self.assertEqual(actualDistribution["total trips"], 304266)  
    self.assertEqual(actualDistribution["min distance"], 0.0)  
    self.assertEqual(actualDistribution["mean distance"], 2.23)  
    self.assertEqual(actualDistribution["max distance"], 66.0)
```

Какие тут будут проблемы?

Unit-тест

```
def test_calculate_distribution_by_valid_taxi_data(self):  
  
    taxiZonesDF = self.spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)  
    taxiDF = self.spark.read.parquet("yellow_taxi_jan_25_2018")  
  
    resultDF = process_taxi_data(taxiDF, taxiZonesDF)  
    actualDistribution = resultDF.collect()[0]  
  
    self.assertEqual(actualDistribution["Borough"], "Manhattan")  
    self.assertEqual(actualDistribution["total trips"], 304266)  
    self.assertEqual(actualDistribution["min distance"], 0.0)  
    self.assertEqual(actualDistribution["mean distance"], 2.23)  
    self.assertEqual(actualDistribution["max distance"], 66.0)
```

Инициализация spark session перед тестом

```
def test_calculate_distribution_by_valid_taxi_data(self):  
  
    taxiZonesDF = self.spark.read.csv("taxi_zones.csv", header=True, inferSchema=True)  
    taxiDF = self.spark.read.parquet("yellow_taxi_jan_25_2018")  
  
    resultDF = process_taxi_data(taxiDF, taxiZonesDF)  
    actualDistribution = resultDF.collect()[0]  
  
    self.assertEqual(actualDistribution["Borough"], "Manhattan")  
    self.assertEqual(actualDistribution["total trips"], 304266)  
    self.assertEqual(actualDistribution["min distance"], 0.0)  
    self.assertEqual(actualDistribution["mean distance"], 2.23)  
    self.assertEqual(actualDistribution["max distance"], 66.0)
```

Unit-тест fixture

```
@classmethod
def setUpClass(cls):
    cls.spark = SparkSession.builder \
        .appName("ТестОбработкиДанныхТакси") \
        .getOrCreate()
```

```
@classmethod
def tearDownClass(cls):
    cls.spark.stop()
```

PyTest: структура, assert и fixture

Какие ещё?

Unit-тест

```
def test_calculate_distribution_by_valid_taxi_data(self):

    taxiZonesDF = self.spark.read.csv("taxi_zones.csv", header=True, inferSchema=True).cache()
    taxiDF = self.spark.read.parquet("yellow_taxi_jan_25_2018").cache()

    resultDF = process_taxi_data(taxiDF, taxiZonesDF)
    actualDistribution = resultDF.collect()[0]

    self.assertEqual(actualDistribution["Borough"], "Manhattan")
    self.assertEqual(actualDistribution["total trips"], 304266)
    self.assertEqual(actualDistribution["min distance"], 0.0)
    self.assertEqual(actualDistribution["mean distance"], 2.23)
    self.assertEqual(actualDistribution["max distance"], 66.0)

    self.taxiZonesDF.unpersist()
    self.taxiDF.unpersist()
```

Очистка кеша между тестами

```
def test_calculate_distribution_by_valid_taxi_data(self):

    taxiZonesDF = self.spark.read.csv("taxi_zones.csv", header=True, inferSchema=True).cache()
    taxiDF = self.spark.read.parquet("yellow_taxi_jan_25_2018").cache()

    resultDF = process_taxi_data(taxiDF, taxiZonesDF)
    actualDistribution = resultDF.collect()[0]

    self.assertEqual(actualDistribution["Borough"], "Manhattan")
    self.assertEqual(actualDistribution["total trips"], 304266)
    self.assertEqual(actualDistribution["min distance"], 0.0)
    self.assertEqual(actualDistribution["mean distance"], 2.23)
    self.assertEqual(actualDistribution["max distance"], 66.0)

    self.taxiZonesDF.unpersist()
    self.taxiDF.unpersist()
```

Unit-тест fixture

```
def setUp(self):
    self.taxiZonesDF = self.spark.read.csv("taxi_zones.csv", header=True,
inferSchema=True)
    self.taxiDF = self.spark.read.parquet("yellow_taxi_jan_25_2018")

def tearDown(self):
    self.taxiZonesDF.unpersist()
    self.taxiDF.unpersist()

def test_calculate_distribution_by_valid_taxi_data(self):
    ... .
```

Итоги видео

- ✓ Научились писать и запускать простые тесты
- ✓ Познакомились с разными способами проверки в тестах
- ✓ Научились создавать fixtures